Predictive Autoscaling for Containerized Applications using Machine Learning

Amit K. Mogal
Department of Computer Science & Applications,
School of Computer-Science and Engineering,
Sandip University,
Nashik, India
amit.mogal@gmail.com

Vaibhav P. Sonaje
Department of Computer Science & Applications,
School of Compute-Science and Engineering,
Sandip University,
Nashik, India
vaibhav.sonaje@sandipuniversity.edu.in

2024 1st International Conference on Cognitive, Green and Ubiquitous Computing (IC-CGU) | 979-8-3503-6178-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/IC-CGU58078.2024.10530773

Abstract: **As containerization continues to gain prominence in modern application deployment, the need for efficient auto-scaling mechanisms becomes paramount to ensure optimal resource utilization and adherence to service level objectives. This research paper proposes a Predictive Auto-scaling framework leveraging machine-learning(ML) techniques to anticipate and proactively adjust the resources allotted to containerized applications. The aim is to enhance the scalability and responsiveness of applications running in dynamic and cloud-native environments. An auto-scaling technique with burst awareness for containerized applications is proposed in this research. This innovative method optimizes container utilization while meeting the Quality-of-Service requirement by taking into account a mix of both vertical and horizontal skills. The proposed proactive autoscaling strategy addresses the challenges posed by unpredictable workloads and varying application demands, fostering adaptability to dynamic environments. By highlighting the significance of machine learning in improving the predictability and effectiveness of resource scaling for containerized systems, this study adds to new work of research on auto-scaling methodologies.**

Keywords - kubernetes, auto-scaling, machine learning, workload forecasting, cloud computing

I. INTRODUCTION

In recent years, containerized applications, facilitated by technologies like Docker and Kubernetes, have revolutionized the deployment and orchestration of software systems. Containerization provides a lightweight and portable solution for packaging, distributing, and running applications, offering scalability and resource efficiency. However, the dynamic nature of modern applications demands adaptive infrastructure management strategies to optimize resource utilization and ensure responsive performance. In this context, auto-scaling emerges as a important paradigm for adjusting the number of application instances based on varying workloads at runtime. Authors [1][2] both emphasize the importance of scalability in cloud computing, with [1]focusing on software-oriented approaches and proposes service replication and migration as effective scalability assurance schemes,while [2] on request scheduling and provides a theoretical definition of scalability in request scheduling and conducts a comprehensive analysis of its scalability metric.

The various container orchestrator viz. Kubernetes, Docker Swarm and others addresses the scaling issues by having Horizontal Pod Autoscaler[3] and Vertical Pod Autoscaler[4] mechanisms. The default autoscaler in Kubernetes is being improved by a number of auto-scaling techniques now in use; however, the majority of these efforts are limited to focusing on a single scaling mode at a time. Because each instance has a set number of resources, only horizontal scalability may result in low container utilization, particularly during the low-request time. However, in the event of a bursty workload, vertical scaling alone not enough to guarantee the quality of service (QoS) needs because it will hit the upper bound. Moreover, in order to ensure service performance, burst identification for auto-scalars must be provided. Traditional auto-scaling solutions often rely on reactive mechanisms triggered by predefined thresholds or rules, which may lead to suboptimal performance and resource allocation. As the demand for more intelligent and proactive auto-scaling strategies grows, the integration of machine learning techniques presents a promising avenue. For scalability in cloud computing, using mobile agents to address performance challenges authors in paper[5] introduces an agent-based framework. However, authors[6] at el provides a broader overview of scalability issues in cloud computing, highlighting the need for exploring the further research avenues. The concept of scalability in Kubernetes is explored in several studies. Paper[7] introduces an adaptive autoscaler, Libra, which automatically adjusts resource allocation and manages horizontal scaling. Reference[8] focuses on server scalability, demonstrating that the application of scalability in Kubernetes can improve performance and server response time.

This research paper explores the domain of predictive auto-scaling for containerized applications, leveraging the capabilities of machine learning to forecast future resource demands and proactively scale the infrastructure accordingly. By developing a predictive auto-scaling mechanism, the aim is to enhance the efficiency and responsiveness of containerized applications, ensuring optimal resource allocation while meeting service level objectives. Authors[9] addresses the challenges of database scaling on Kubernetes, proposing a solution for managing highly available databases using auto-scaling.In Reference[10] delves into the operational behaviors of Horizontal Pod Autoscaler (HPA) in Kubernetes, providing insights on its performance optimization. The literature studies collectively enhances the importance of scalability in Kubernetes and the potential for its improvements through various mechanisms.

Kubernetes has a scaling engine that pursues the auto-scaling choices able for taking care of the genuine fluctuation of approaching solicitations, and presents a minimal administration boundary for the cloud-tenant application provider to easily set their sweet spot for over provisioning of the resource vs. Service Level Aggreement violation trade-off[11]. An active Kubernetes autoscaler based on pod replica prediction is proposed to improve resource scaling efficiency, and it shows that the advantage of the autoscaler proposed in research paper[12] is faster response speed. The main

Authorized licensed use limited to: University of Greenwich. Downloaded on October 31,2025 at 15:57:45 UTC from IEEE Xplore. Restrictions apply.

concerned of our research lies in its potential to advance the state of auto-scaling strategies and methods use for containerized applications by introducing a proactive and data-driven approach.

To address the aforementioned issues, we suggest in this paper a hybrid scaling approach for Kubernetes containerized applications as well as a horizontally autoscaling pods with variable pods. The main contributions of proposed reserach are : provide a rolling update deployment methodology when varying in two ways one the number of pods and the other is resources used per pod for scaling execution to prevent service disruption; this different way of hybrid auto-scaling method aims by satisfying QoS to optimize the use of resources for containers within Kubernetes. In our proposed research we combined the predictive autoscaling method and indetifying burst to improve decisions in resources scaling is new ways.

Further, this research is structured as follows: second part mentions the related research work. We present our auto-scaling solution in third part. The challenges and future research directions are described inpart four. In fifth part we conclude the research.

II. RELATED RESAERCH

In this part, we present recent works that are close to our proposal. We mainly focus on research related to container sampling method in Kubernetes environment. We found that research papers[13] and [14] both address the issue of workload forecasting and burst identification in Kubernetes. Authors[13] at.el. focuses on the performance of a serverless architecture on Kubernetes, identifying the breaking point and the need for scaling whereas authors [14] at. el. proposes a proactive custom autoscaler for workload prediction by using a machine learning model, which outperforms default horizontal pod auto scaler of kubernetes. These studies highlight the importance of accurate workload forecasting and efficient burst identification in Kubernetes, and the potential of advanced techniques such as deep learning for addressing these challenges.

The research paper[15] authors mentioned a proactive method for vertical scaling to predict the resorces utilization viz. CPU for improving scaling decisions to optimize the pod resource by using the statistical method Holt-Winters exponential smoothing and Long Short-Term Memory a deep learning model. We contend that there has the QoS-poorlying constraint further. First off, scaling indicators that are unable to accurately reflect the QoS requirements of apps can only be done by vertical scaling when working with low-level metrics (CPU, memory). Second, they omitted to discuss the detrimental impact of a service interruption.

Reference[16] and [17] presents ‘an appropriate continuum deployment’ method based-on customised kubernetes controllers for Continuous Integration and Continuous Delivery, known as ‘Kontinuum Controller’, and investigates its scalability with focus on reconciling adjusted configuration settings per application and per node. Although conventional HPA is outperformed by these proactive horizontal scaling techniques because they can quickly adjust to meet QoS requirements, the main issue with the aforementioned research is that they are unable to fully optimize resource use, particularly during low-request periods. The issue is because during scaling procedures, the horizontal scaling type is unable to modify both the resource for each instance and their quantity.

Vertical scaling is a disadvantage when it comes to preventing SLO violations, but it has an advantage when it comes to optimizing resource use. Even though horizontal scaling can effectively meet QoS requirements, it has the disadvantage of simply optimizing pod consumption. Achieving the hybrid scaling strategywhich benefits from both vertical and horizontal scaling methodsis essential.No study supports the hybrid scaling approach in the Kubernetes environment, where decisions on hybrid scaling are made concurrently for the application's run-time deployment. In order to meet the QoS requirement and maximize resource consumption, we therefore suggest the hybrid auto-scaling strategy, which can alter both the pods number and the resources use per pods. Motivated by great success of deep learning, we deal with the varying workload by combining burst identification with the Bidirectional Long/Short Term Memory (Bi-LSTM) model for enhancing the scaling decisions.

III. PROPOSED RESEARCH

Our study combines a burst recognition module with a prediction method based on ML methodology to estimate the application's future demand and improve the efficacy of scaling decisions. Keep in mind that burst identification algorithms can be much more complex, depending on the nature of your data and the specific requirements of application. The algorithm1 serves as a basic illustration, and you may need to customize it based on use case.

A. Forecasting workload

For many years, researchers in a variety of application domains have focused on short-term demand forecasting, and machine learning approaches have proliferated, particularly in time-series prediction. A solution for auto-scaling Kubernetes worker nodes to support dynamic workloads[18], while [19] explained fine-grained scheduling policies for containerized High Performance Computing(HPC) workloads in clusters at kubernetes. With the objective to increase elasticity and resilience authors of research paper[20]make practical demonstrations of the application in Kubernetes Clusters Federation in Fog-Computing, and reference[13] proposes a fault tolerant serverless architecture that based on Kubernetes for handling realistic workloads. These studies collectively highlight the potential of Kubernetes clusters in managing diverse workloads, and the need for further research in this area. A realistic scaled down Azure workload traces dataset was used to test an open-source framework-based, production-grade, and fault tolerant serverless architecture running on OpenStack instances and based on a high available kubernetes topologies[13]. It is challenging for statistical models to forecast the varying workloads, though. More sophisticated deep learning-based

Authorized licensed use limited to: University of Greenwich. Downloaded on October 31,2025 at 15:57:45 UTC from IEEE Xplore. Restrictions apply.

techniques perform better when working with non-stationary and non-linear time-series datasets.

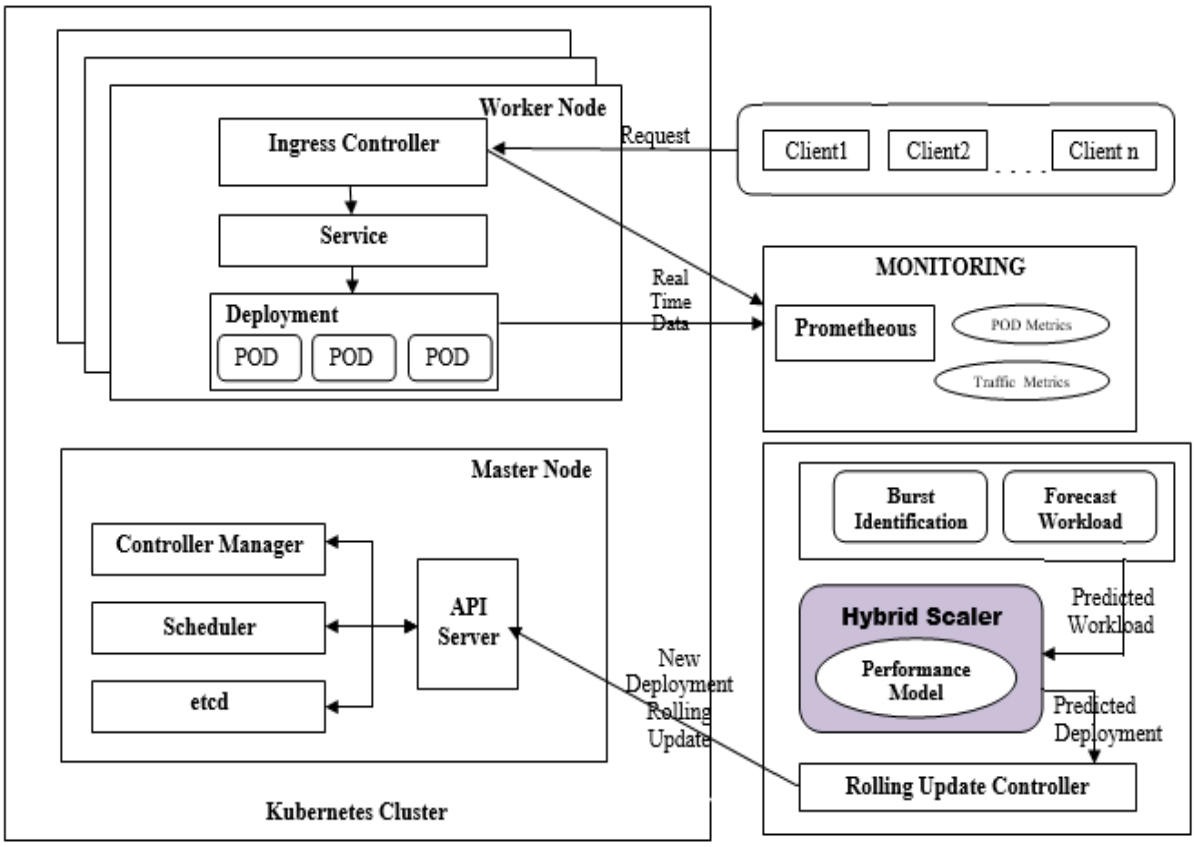

Fig 1 : The Propose System Architecture for Hybrid AutoScaler

B. Identifying bursts

Since we offer the hybrid scaling technique, our work's burst detection is more precise because it is based on the anticipated workload. The workload is considerable and varies drastically in burst conditions. As a result, in our work, we prefer to keep a pod's maximum resources and horizontally scale the pods to swiftly respond to burst scenarios; in other cases like to fulfill QoS, we optimize pod use while adjusting the pods and the resources with pod. We do not distinguish between other burst kinds in this paper, such as bursts of varying lengths or intensities. Only when the workload is expected to change rapidly in order to meet QoS requirements do we directly provide the burst signal. In reference [21] authors looked into the best way to detect bursts online and suggested a new predictive auto-scaling way to meet certain response time requirements. The method was then compared to several cutting-edge baseline autoscaled methods under a variety of synthetic and realistic bursty workloads for sample application.

Below mention Algorithm1 for identifying burst at realtime assumes a threshold for burst detection. If a data point exceeds the threshold, it starts or extends a burst. If the data point is below the threshold, the burst level decays over time. Authors at el [22] offer a novel averages-based model and metric that demonstrate the ability to identify, quantify, and predict burst behavior in workloads without relying on the underlying base functions. This model may be applied to any type of burst prediction for time series. In order to address these issues, a bursty workloads generator which based on the two-state Markovian Arrival Process (MAP2) has been presented in paper [23] for Cloudstone, a cloud benchmark. The probability density function can be use to predicte the use of C.P.U. by method called 'fine-grained performance analysis', which has been recommended for cloud-based applications to assist improved resource provisioning, decision making and optimising system performance.

Our real-time burst identification algorithm is displayed in Algorithm1. The __init__ method initializes the object with default or user-specified values for threshold and decay_factor.self.current_value is set to 0 initially. If there is no ongoing burst (current burst level is less than or equal to 0), it starts a new burst.If there is an ongoing burst, it extends the current burst. The burst decay process begins if the current data point is not larger than the threshold. Function start_burst(self) prints a message indicating that a burst has started and sets self.current_value to 1 whereas function extend_burst(self)prints a message indicating that the burst has extended and increments the burst level.Next, we determine if there is an ongoing burst (current burst level is greater than 0), decay_burst(self) function prints a message indicating that the burst has decayed and decreases the burst level by decay_factor. At last step we create Creates an instance of OnlineBurstIdentification and then simulates a series of data points (data_points) and processes each data

Authorized licensed use limited to: University of Greenwich. Downloaded on October 31,2025 at 15:57:45 UTC from IEEE Xplore. Restrictions apply.

point using the process_data_point method of the burst_identifier object.In summary, this algorithm code defines a class that can identify, start, extend, and decay bursts based on a threshold and a decay factor. The example usage demonstrates how the class can be employed to process a sequence of data points and track burst events.

```
class OnlineBurstIdentification:
    def __init__(self, threshold=3, decay_factor=0.1):
            self.threshold = threshold
            self.decay_factor = decay_factor
            self.current_value = 0
    def process_data_point(self, data_point):
            if data_point > self.threshold:
                    # Check if there is an ongoing burst
                        if self.current_value <= 0:
                                self.start_burst()
                        else:
                            self.extend_burst()
            else:
                    # Decay burst level if no burst is ongoing
                      self.decay_burst()
    def start_burst(self):
            print("Burst started!")
            self.current_value = 1

    def extend_burst(self):
            print("Burst extended!")
            self.current_value += 1

    def decay_burst(self):
            if self.current_value > 0:
                    print("Burst decayed!")
                    self.current_value -= self.decay_factor
# Example usage:
   burst_identifier = OnlineBurstIdentification()
            # Simulate data points
   data_points = [2, 4, 1, 5, 6, 3, 2, 1, 4, 7, 8]
   for data_point in data_points:
            burst_identifier.process_data_point(data_point)
```

Algorithm 2 : Pseudo code for identifying burst at realtime (python)

IV. Machine Laerning Autoscaling Method

We present our approach to provide the hybrid scaling strategy in Fig 3Proposed hybrid auto-scalingAlgorithm.The hybrid autoscaling algorithm involves several components, including performance modeling, prediction using a forecasting model, and decision-making based on response time thresholds and resource utilization. The forecasting modelA model used for predicting future response time and utilization. Function _init_(self) is the constructor initializes an empty list (self.data) to store historical response time and utilization data.The initial resource allocation per pod (set to 1.0). A monitor_and_scale function monitors the current response time and utilization and decision will be made whether to scale up or scale down based on predictions make from the forecast model.It updates
the forecast model with current data and checks if scaling actions are needed. The oscillation issue causes the scalar to regularly execute opposing activities in a brief amount of time [25], wasting money and resources. Hence, when the anticipated reaction time over the threshold, the function scale_up(self) is called. It increases the pods number and/or resource allocated for every pod while function scale_down is called when the predicted utilization is below the target. It not only decreases the resource allocation per pod but also the numebr of pods; but not below one(1) pod.Based on the anticipated workload, our system will use the burst signal at each time period. When there is no burst, we adjust the deployment to maintain response times below the threshold by adjusting pods numbers and resources utilization per pod. In research paper [24] author at el. Suggest that when the need of resources reaches a predefined value termed as threshold, its capacity will not get remarkably improvised. Forecasting model help in updates the pods required and resources thar needed for pod based on response-time and utilization predictions. In the last step, for loop simulates the monitoring of response time and utilization over ten iterations. It generates random values for response time and utilization and calls the monitor_and_scale function of the autoscaler to make scaling decisions. In summary, pseudo code outlines a basic implementation of a hybrid autoscaling algorithm using a forecasting model. The autoscaler helps to adjusts the

Authorized licensed use limited to: University of Greenwich. Downloaded on October 31,2025 at 15:57:45 UTC from IEEE Xplore. Restrictions apply.

number of pods and resources allocated based on workload-predictions and predefined thresholds.Creating a complete new auto-scaling algorithm involves several components, including performance modeling, prediction using a forecasting model, and decision-making based on response time thresholds and resource utilization. Given the complexity of such an algorithm, We provide a simplified method that outlines the main steps. Keep in mind that a real-world implementation may require more sophistication and customization based on your specific needs.

```
import numpy as np
class HybridAutoscaler:
    def __init__(self, response_time_threshold, target_utilization, forecast_model):
        self.response_time_threshold = response_time_threshold
        self.target_utilization = target_utilization
        self.forecast_model = forecast_model
        self.current_pods = 1  # Start with one pod
        self.current_resource_per_pod = 1.0  # Start with a default resource per pod
    def monitor_and_scale(self, current_response_time, current_utilization):
        # Update the forecast model with the current data
  self.forecast_model.update_data(current_response_time, current_utilization)
        # Get the predicted values from the forecast model
   predicted_response_time, predicted_utilization = self.forecast_model.predict()
# Check if the predicted response time exceeds the threshold
   if predicted_response_time > self.response_time_threshold:
# Scale up if response time is too high
       self.scale_up()
 elif predicted_utilization < self.target_utilization:
            # Scale down if predicted utilization is below the target
    self.scale_down()
    def scale_up(self):
# Increase the number of pods and/or resource per pod
   self.current_pods += 1
   self.current_resource_per_pod += 0.1  # Adjust based on your requirements
   print(f"Scaling up: {self.current_pods} pods, {self.current_resource_per_pod} resource per pod")
    def scale_down(self):
        # Decrease the number of pods and/or resource per pod
    if self.current_pods > 1:
        self.current_pods -= 1
      self.current_resource_per_pod -= 0.1  # Adjust based on your requirements
      print(f"Scaling down: {self.current_pods} pods, {self.current_resource_per_pod} resource per pod")

# Example usage:
class ForecastModel:
    def __init__(self):
     self.data = []
    def update_data(self, response_time, utilization):
   self.data.append((response_time, utilization))
    def predict(self):
        # Simple prediction for illustration purposes
        if len(self.data) > 1:
     last_response_time, last_utilization = self.data[-1]
    predicted_response_time = last_response_time + np.random.normal(0, 0.1)
    predicted_utilization = last_utilization + np.random.normal(0, 0.1)
    return predicted_response_time, predicted_utilization
        else:
     return 1.0, 0.5  # Default values for the first prediction
# Initialize the autoscaler with a response time threshold, target utilization, and forecast model
autoscaler=HybridAutoscaler(response_time_threshold=2.0,target_utilization=0.8, forecast_model=ForecastModel())
# Simulate monitoring and scaling decisions
for _ in range(10):
  current_response_time = 2.5 + np.random.normal(0, 0.1)
  current_utilization = 0.7 + np.random.normal(0, 0.1)
 autoscaler.monitor_and_scale(current_response_time, current_utilization)
```

Algorithm 2: Pseudo code for proposed hybrid auto-scaling algorithm (python)

For proposed algorithm of hybrid autoscaling we assume a simplified forecasting model and update the number_of_ pods and/or resources_per_pod based on response-time and utilization predictions. In a real-world scenario, you would need to implement a more accurate forecasting model and fine-tune the scaling decisions based on your specific use case and performance characteristics.

V. Future Research Direction

While the current research demonstrates the efficacy of machine learning models in predicting resource demands for auto-scaling containerized applications, a promising avenue for future investigation involves the integration of reinforcement learning techniques to enhance the applicability and responsiveness of autoscaling methods. Machine learning can be employed to develop dynamic and self-learning auto-scaling policies that continuously adapt to changing workload patterns, system dynamics, and

Authorized licensed use limited to: University of Greenwich. Downloaded on October 31,2025 at 15:57:45 UTC from IEEE Xplore. Restrictions apply.

application behavior. Unlike static models, reinforcement learning algorithms can optimized auto-scaling to make decisions over time based on realtime feedback, considering both short-term and long-term objectives. By venturing into the realm of machine learning, researchers can contribute to the development of intelligent auto-scaling mechanisms that have the potentiality to enhance the scalability, efficiency, and reliability of containerized applications in dynamic and heterogeneous computing environments. Extend the research to handle multiple objectives concurrently, such as minimizing response time, optimizing cost, and maximizing resource utilization. Design a reward function that captures the trade-offs between these objectives to achieve a balanced auto-scaling policy. Address the exploration-exploitation dilemma inherent in reinforcement learning. Balance the need to explore new scaling strategies against exploiting known policies to ensure optimal decision-making in varying conditions. This is particularly relevant for containerized applications deployed across diverse infrastructure environments.

VI. CONCLUSION

Current predictive auto-scaling models often rely on historical data and supervised learning to make scaling decisions. In future we extend research in exploring the integration of reinforcement learning (RL) techniques to improves the scalability and adaptability of the autoscaling system. However, RL introduces the ability to learn optimal scaling policies through interaction with the environment. The future research will evaluate the performance of the proposed predictive-auto-scaling approch through extensive experimentation and benchmarking across diverse containerized application scenarios. The results and evaluation will showcase the framework's ability to optimize resource allocation, mitigate performance bottlenecks, and maintain service quality within defined thresholds. Additionally as extension research, one can explores the trade-offs between accuracy and latency in the prediction models, providing insights into the practical implementation of predictive auto scaling in real-world applications. The findings presented in this paper offer valuable insights for practitioners and researchers aiming to improve the scalability and responsiveness of containerized deployments in dynamic and cloud-native environments.

REFERENCES

[1] Lee, Yoo and Kim. "Software Approaches to Assuring High Scalability-in-Cloud-Computing". *2010 IEEE 7thInternational Conference on E-Business Engineering*.

[2] Xue, Chao et al. "Scalability analysis of request scheduling in cloud computing". *Tsinghua Science_and_Technology* (2019)

[3] Horizontal Pod Autoscsaling in Kubernetes : https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/, [Accessed on: December. 27, 2023].

[4] Vertical Pod Autoscsaling in Kubernetes : https://github.com/kubernetes/autoscaler/tree/master/verticalpod-autoscaler, [Accessed on: December. 27, 2023].

[5] Singh, Aarti and Malhotra. "Agent Based Framework for Scalability in Cloud Computing".

[6] Somasundaram, Thamarai Selvi et al. "Scalability issues in cloud computing". *2012 Fourth International Conference on Advanced Computing (ICoAC)*.

[7] Balla, David et al. "Adaptive scaling of Kubernetes pods." *NOMS 2020 - 2020 IEeE/IFIP Network Operations and Management Symposium*.

[8] Dewi, Lily Puspa et al. "Server Scalability Using Kubernetes". *2019 4th Technology Innovation Management and Engineering Science International Conference*.

[9] Perera, H. C. S. et al. "Database Scaling on Kubernetes". *2021 Third International Conference on Advancements in Computing (ICAC)* (2021).

[10] Nguyen, Thanh-Tung et al. "Horizontal Pod Auto-scaling in Kubernetes for Elastic Container Orchestration." *Sensors (Basel, Switzerland)* 20.

[11] Toka, László et al. "Machine Learning-Based Scaling Management for Kubernetes Edge Clusters". *IEEE Transactions on Network and Service Management* 18.

[12] Hu, Tengfei and Yannian Wang. "A Kubernetes Autoscaler Based on Pod Replicas Prediction." *2021 Asia Pacific Conference on Communications Technology & Computer Science (ACCTCS)* (2021): 238 - 241.

[13] Govind, Hima and Horacio González-Vélez. "Benchmarking Serverless Workloads on Kubernetes." *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)* (2021): 704 -712.

[14] Dang-Quang, Nhat-Minh and Myungsik Yoo. "Deep Learning-Based Auto-scaling Using Bidirectional Long Short-Term Memory for Kubernetes". *Applied Sciences*.

[15] T. Wang, S. Ferlin, & M. Chiesa, "Predicting CPU usage for proactive auto-scaling" in *Proceeding. First Workshop on Machine Learning and Systems*.

[16] Hass, Daniel and Josef Spillner. "Workload Deployment and Configuration Reconciliation at Scale in Kubernetes-Based Edge-Cloud Continuums". *21st International Symposium on Parallel and Distributed Computing (ISPDC)*.

[17] C. Qu, R. N. Calheiros, and R. Buyya, "Auto-scaling web applications in clouds: A taxonomy and survey". *ACM Computing Surveys (CSUR)*, volume 51.

[18] Thurgood, Brandon and Ruth G. Lennon. "Cloud Computing With Kubernetes Cluster Elastic Scaling". Proceedings of the 3rd International Conference on Future Networks and Distributed Systems.

[19] Liu, Peini and Jordi Guitart. "Fine-Grained Scheduling for Containerized HPC Workloads in Kubernetes Clusters". IEEE 24th Int. Conf on High Performance Computing & Communications; 8th Int. Conf on Data Science & Systems; 20th International Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application.

[20] Faticanti, Francescomaria et al. "An Application of Kubernetes Cluster Federation in Fog Computing". 24th Conference on Innovation in Clouds, Internet and Networks and Workshops (ICIN) (2021): 89-91.

[21] Tahir, Fatima et al. "Online Workload Burst Detection for Efficient Predictive Auto-scaling of Applications". IEEE(2020) 73730 - 73745.

[22] Lassnig, Mario et al. "Identification, Modelling and Prediction of Non-periodic Bursts in Workloads".10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing (2010): 485-494.

[23] Yin, Jianwei et al. "System resource utilization analysis and prediction for cloud based applications under bursty workloads". Inf. Sci. 279 338-357.

[24] F. Rossi, "Auto-scaling policies to adapt the application deployment in Kubernetes,'' in Proceeding. ZEUS, pp. 30–38.

[25] KEPS: in-place update of pod resources [Online]. Available: https://github.com/kubernetes/enhancements/tree/master/keps/sig-node/1287-in-place-update-pod-resources,[Accessed on: December.29, 2023].

Authorized licensed use limited to: University of Greenwich. Downloaded on October 31,2025 at 15:57:45 UTC from IEEE Xplore. Restrictions apply.